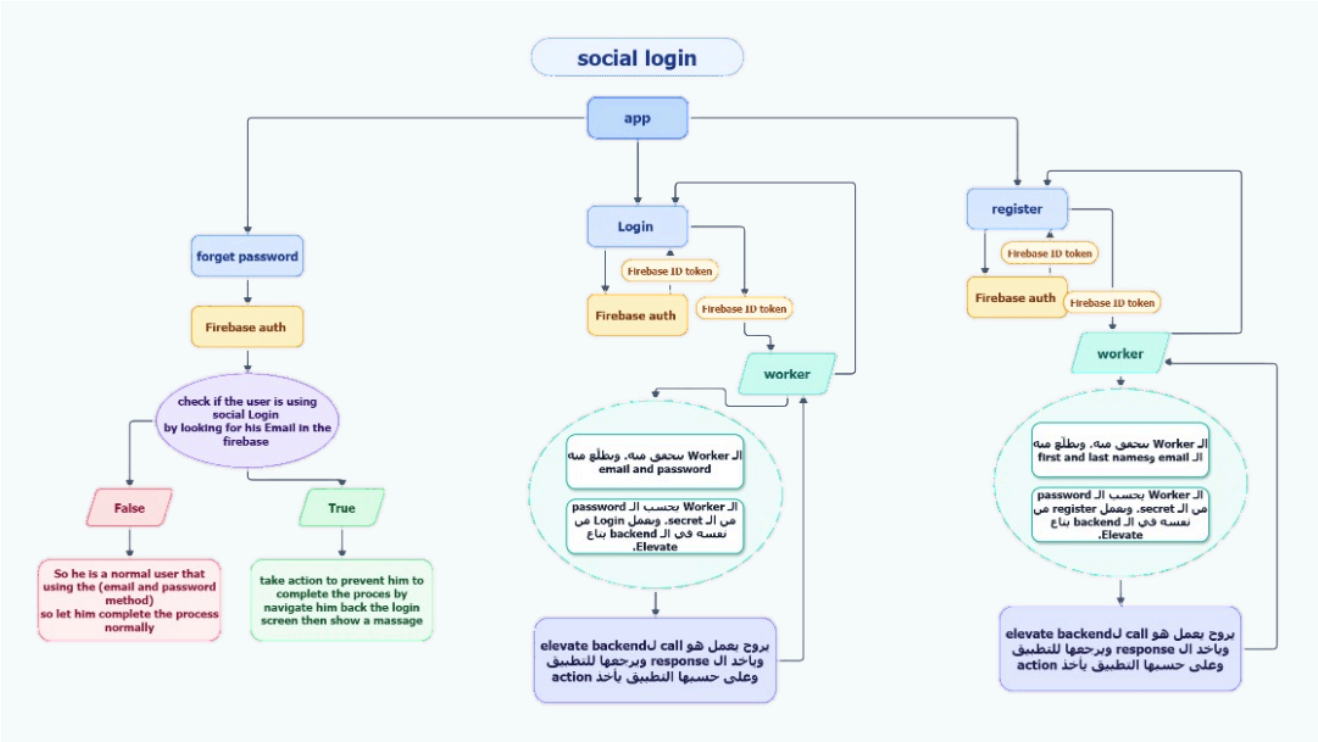
الدليل التقني الشامل: نظام تسجيل الدخول الاجتماعي (Social Login)

توثيق معماري وعملي لربط تطبيق Flutter مع Firebase Auth وCloudflare Workerg وواجهة Elevate Backend

نظرة عامة على المعمارية وتدفق البيانات (Architecture Overview)

يوضح هذا الدليل الخطوات الكاملة لإعداد وتشغيل منظومة تسجيل الدخول الاجتماعي (Google Sign-In) في تطبيق Flutter وربطها مع الواجهة الخلفية الحالية (Legacy Backend) دون الحاجة إلى تعديل كود السيرفر القديم، وذلك عبر بوابة وسيطة آمنة مبنية على **Cloudflare Worker**.



شكل (1): المخطط المعماري الكامل لمسارات التطبيق الثلاثة (Login، Register، Forget Password) وعلاقتها بـ Firebase Auth وCloudflare Workerg.

المسار (Flow)	الجهة المسؤولة	آلية العمل المختصرة	النتيجة النهائية
تسجيل الدخول (Login)	Flutter + Firebase Auth + Worker	المصادقة عبر Google واستخراج `Firebase ID token`، ثم التحقق منه في الـ Worker واشتقاق الباسورد من `SERVER_SECRET` والاتصال بـ `/api/v1/auth/signin`.	إرجاع بيانات المستخدم وتوكن الـ Backend أو حالة `not_registered` (404).

المسار (Flow)	الجهة المسؤولة	آلية العمل المختصرة	النتيجة النهائية
إنشاء حساب (Register)	Flutter + Firebase Auth + Worker + Firestore	جمع بيانات الملف الشخصي وإرسالها مع `Firebase ID token` إلى الـ Worker `/api/v1/auth/signup` للتسجيل في ، ثم حفظ البريد في مجموعة `social_users` .	إنشاء الحساب في Elevate Backend وتوثيق نوع المزود `google` في Firestore.
نسيت كلمة المرور (Password Forgot)	Flutter + Cloud Firestore	فحص البريد الإلكتروني داخل مجموعة `social_users` في Firestore قبل إرسال طلب الـ OTP للسيرفر القديم.	منع حسابات Google من تغيير الباسورد وتوجيهها لزر الدخول الاجتماعي، والسماح للحسابات العادية بالمتابعة.

الجزء 1: إعداد Firebase

1. اعمل مشروع

- افتح `console.firebase.google.com` .
- اختر **Create a project** ← اكتب اسم المشروع ← كمّل الخطوات (تقدر تقفل Google Analytics) ← اضغط **Create**.

2. هات الـ Project ID

- من القائمة الجانبية افتح ⚙️ **Project settings** ← تبويب **General** ← انسخ قيمة **Project ID** (مش الـ Project name). هتحتاجه في إعدادات الـ Worker.

3. فعّل Google Sign-in

- ادخل على **Authentication** ← اضغط **Get started**.
- اختر تبويب **Sign-in method** ← اضغط على **Google** ← فعّل خيار **Enable**.
- اكتب **Public-facing name** (اسم التطبيق) واختار **Support email**.
- اضغط **Save**.
- افتح **Google** تاني ← افتح قسم **Web SDK configuration** ← انسخ **Web client ID**. هتحتاجه في كود Flutter.
- *ملاحظة:* لو الحقول دي مقفولة، يبقى لسه ماعملتش Enable وحفظت الإعدادات.

4. اعمل Firestore

- ادخل على **Firestore Database** ← اضغط **Create database** ← اختار الـ location المناسب ← اختر **Production mode**.
- افتح تاب **Rules** ← امسح اللي موجود بالكامل وحط القواعد التالية:

firestore.rules Copy

```
rules_version = '2';
service cloud.firestore {
```

```
  match /databases/{database}/documents {
    match /social_users/{email} {
      allow get: if true;
      allow create: if request.auth != null
                    && request.auth.token.email_verified
                    && request.auth.token.email.lower() == email;
      allow list, update, delete: if false;
    }
  }
}
```

- اضغط **Publish** لحفظ وتفعيل القواعد.
- *ملاحظة:* الـ collection اسمه `social_users` وبيتعمل لوحده تلقائياً أول ما التطبيق يكتب فيه.

الجزء 2: الـ Worker (Cloudflare Worker)

1. اعمل المشروع

نفّذ الأمر التالي لإنشاء مشروع Cloudflare Worker جديد:

Terminal / Command Prompt Copy

```
npm create cloudflare@latest social-gateway
```

خلال خطوات الإنشاء التفاعلية اختار الخيارات التالية:

- **What would you like to start with?** ← `Hello World Worker`
- **Which language do you want to use?** ← `TypeScript`
- **Do you want to use git for version control?** ← أي حاجة (Yes أو No)
- **Do you want to deploy your application?** ← `No`

بعد إنشاء الفولدر، ادخل جواه وثبّت مكتبة `jose` للتحقق من توكنات JWT:

Terminal / Command Prompt Copy

```
cd social-gateway
npm install jose
```

2. ملف الإعدادات wrangler.jsonc

افتح ملف `wrangler.jsonc` واستبدل محتواه بالكامل بالكود التالي:

wrangler.jsonc Copy

```
{
  "$schema": "node_modules/wrangler/config-schema.json",
  "name": "social-gateway",
  "main": "src/index.ts",
  "compatibility_date": "2026-10-19",
```

```
  "observability": { "enabled": true },
  "upload_source_maps": true,
  "workers_dev": true,
  "preview_urls": false,
  "vars": {
    "LEGACY_URL": "https://fitness.elevateegy.com",
    "FIREBASE_PROJECT_ID": "هنا-الـ-Project-ID"
  }
}
```

- `LEGACY_URL`: رابط الـ backend القديم، من غير `/api/v1` ومن غير `/` في الآخر.
- `FIREBASE_PROJECT_ID`: المعرّف اللي نسخته من إعدادات مشروع Firebase.
- حدد اسم الـ Worker (`name`) قبل أول deploy. لو اتغيّر بعد كده بيتعمل Worker جديد بلينك جديد.
- لو الملف ضارب error، راجع إن مفيش فاصلة ناقصة أو زيادة.

3. الكود البرمجي للـ Worker في src/index.ts

افتح ملف `src/index.ts` واستبدل محتواه بالكامل بالكود التالي:

src/index.ts (TypeScript) Copy

```
import { createRemoteJWKSet, jwtVerify } from 'jose';

interface Env {
  LEGACY_URL: string;
  FIREBASE_PROJECT_ID: string;
  SERVER_SECRET: string;
}

const JWKS = createRemoteJWKSet(
  new
URL('https://www.googleapis.com/service_accounts/v1/jwk/securetoken@system.gserviceaccou
nt.com')
);

const json = (data: unknown, status = 200) =>
  new Response(JSON.stringify(data), { status, headers: { 'Content-Type':
'application/json' } });

const enc = (s: string) => new TextEncoder().encode(s);

async function derivePassword(secret: string, email: string) {
  const key = await crypto.subtle.importKey('raw', enc(secret), { name: 'HMAC', hash:
'SHA-256' }, false, ['sign']);
  const sig = await crypto.subtle.sign('HMAC', key, enc('social:' +
email.toLowerCase()));
  const b64 = btoa(String.fromCharCode(...new Uint8Array(sig))).replace(/[+/=]/g,
'').slice(0, 32);
  return `Aa1!${b64}`;
}

const callLegacy = (env: Env, path: string, body: object) =>
  fetch(`${env.LEGACY_URL}/api/v1/auth/${path}`, {
    method: 'POST',
```

```
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(body),
  });

async function verify(token: string, env: Env) {
  const { payload } = await jwtVerify(token, JWKS, {
    issuer: `https://securetoken.google.com/${env.FIREBASE_PROJECT_ID}`,
    audience: env.FIREBASE_PROJECT_ID,
  });
  if (!payload.email || !payload.email_verified) throw new Error('email not verified');
  return String(payload.email).toLowerCase();
}

export default {
  async fetch(req: Request, env: Env): Promise<Response> {
    const { pathname } = new URL(req.url);
    if (req.method !== 'POST') return json({ message: 'not found' }, 404);

    const body = await req.json<any>().catch(() => ({} as any));
    if (!body.token) return json({ message: 'token required' }, 400);

    let email: string;
    try {
      email = await verify(body.token, env);
    } catch {
      return json({ message: 'invalid token' }, 401);
    }
    const password = await derivePassword(env.SERVER_SECRET, email);

    // POST /social/login  { token }
    if (pathname === '/social/login') {
      const res = await callLegacy(env, 'signin', { email, password });
      if (res.ok) return new Response(res.body, { status: 200, headers: { 'Content-
Type': 'application/json' } });
      return json({ message: 'not_registered' }, 404);
    }

    // POST /social/register  { token, profile: {...} }
    if (pathname === '/social/register') {
      if (!body.profile) return json({ message: 'profile required' }, 400);
      const res = await callLegacy(env, 'signup', { ...body.profile, email, password,
rePassword: password });
      return new Response(res.body, { status: res.status, headers: { 'Content-Type':
'application/json' } });
    }

    return json({ message: 'not found' }, 404);
  },
};
```

شكل الـ API القديم اللي الكود بيكلمه:

المسار (Endpoint)	البيانات المرسلة (Request Body)	الاستجابة (Response)
POST /api/v1/auth/signin	{ email, password }	{ message, user, token }
POST /api/v1/auth/signup	{ firstName, lastName, email, password, rePassword, gender, height, weight, age, goal, activityLevel }	{ message, user, token }

4. انشر (Deploy)

Terminal / Command Prompt Copy

```
npx wrangler login
npx wrangler deploy
```

- الأمر `login` بيفتح المتصفح، اضغط موافقة لتسجيل الدخول.
- أول deploy على الحساب هيسألك تسجّل `workers.dev subdomain`. اكتب اسم عام للفريق (مثلًا `fitness-app`، ولو اتاخد جرّب غيره). ده بيتسجل مرة واحدة للحساب.
- في الآخر هيطلعلك اللينك النهائي:

Worker Endpoint URL Copy

```
https://social-gateway.<subdomain>.workers.dev
```

- انسخه؛ ده الـ **Base URL** اللي التطبيق هيكلمه.
- **تنبيه للفريق**: واحد بس في الفريق يعمل الـ deploy، والباقي يستخدموا اللينك مباشرة.

5. الـ Secret (مهم جدًا)

الـ secret ده مفتاح توليد الـ password بتاع كل حسابات الـ social. انت اللي بتولّده بنفسك:

Terminal / Command Prompt Copy

```
node -e "console.log(require('crypto').randomBytes(48).toString('base64'))"
```

انسخ الناتج واحفظه في password manager، وبعدين ارفعه للـ Worker بالأمر:

Terminal / Command Prompt Copy

```
npx wrangler secret put SERVER_SECRET
```

الصق القيمة لما يسألك في الـ terminal.

- **تحذير أمني**: متكتبوش في Git ولا في الشات ولا في الـ Documentation.
- لو ضاع أو اتغيّر، كل حسابات الـ social المسجلة سابقاً هتبوظ. خد منه نسخة احتياطية آمنة.

- لو في المستقبل حد تاني هيعمل deploy، لازم يبقى معاه الـ secret (الـ deploy العادي مش بيمسحه).

6. اتأكد إنه شغال

Terminal / Command Prompt Copy

```
npx wrangler secret list
```

لازم تلاقي `SERVER_SECRET` ظاهر في القائمة. وبعدين اختبر الـ endpoint بـ `curl`:

Terminal / Command Prompt Copy

```
curl -X POST https://social-gateway.<subdomain>.workers.dev/social/login -H "Content-
Type: application/json" -d "{\"token\":\"test\"}"
```

المفروض يرد `{"message":"invalid token"}`. ده معناه إن الـ Worker شغّال ومتصل بشكل سليم.

الجزء 3: ربط مشروع Flutter بـ Firebase

1. الأدوات (مرة واحدة على الجهاز)

Terminal / Command Prompt Copy

```
npm install -g firebase-tools
dart pub global activate flutterfire_cli
firebase login
```

في `firebase login` اختار نفس حساب Google اللي فيه المشروع. لو عندك أكتر من حساب وظهرلك مشاريع غلط نفّذ:

Terminal / Command Prompt Copy

```
firebase logout
firebase login
```

2. الـ packages

من فولدر مشروع الـ Flutter (اللي فيه `pubspec.yaml`) نفّذ:

Terminal / Command Prompt Copy

```
flutter pub add firebase_core firebase_auth google_sign_in dio cloud_firestore
```

3. اربط المشروع

Terminal / Command Prompt Copy

```
flutterfire configure
```

- اختار مشروع Firebase بتاعك من القائمة.
- اختار **Android** بس (اضغط Space على المنصات اللي مش عايزها عشان تتشال، واضغط Enter للتأكيد).
- ده بيسجّل التطبيق في Firebase بالـ package name الصح، وبيعمل ملف `lib/firebase_options.dart` وملف `google-services.json`.

4. تهيئة main.dart

lib/main.dart (Dart) Copy

```
import 'package:firebase_core/firebase_core.dart';
import 'package:flutter/material.dart';
import 'firebase_options.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(options: DefaultFirebaseOptions.currentPlatform);
  runApp(const MyApp());
}
```

الجزء 4: الـ SHA (ضروري لـ Google Sign-in على Android)

من غير إضافة بصمات الـ SHA، عملية الـ Google sign-in هتفشل على أجهزة ومحاكي Android.

1. شغّل التطبيق مرة على Android

Terminal / Command Prompt Copy

```
flutter run
```

تشغيل التطبيق لأول مرة بيعمل ملف `debug.keystore` تلقائياً لو مش موجود.

2. اطلع الـ SHA

محتاج Java 17 أو أحدث، والأسهل تستخدم النسخة الموجودة جوه Android Studio. استبدل المسار بمكان Android Studio عندك:

Terminal / Command Prompt Copy

```
set "JAVA_HOME=<مكان-Android-Studio>\jbr"
set "PATH=%JAVA_HOME%\bin;%PATH%"
keytool -list -v -keystore "%USERPROFILE%\.android\debug.keystore" -alias
androiddebugkey -storepass android -keypass android
```

- انسخ سطر **SHA1** وسطر **SHA256**.
- *ملاحظة:* مكان Android Studio لو مش عارفه، شغّل `flutter doctor -v` ودوّر على سطر `Java binary` وشوف المسار بعد `at:`.

3. ضيفهم في Firebase

- ادخل على ⚙️ **Project settings** ← قسم **Your apps** ← تطبيق الـ Android ← اضغط **Add fingerprint**.
- الصق **SHA1**، دوس زرار SHA1، ثم **Save**.
- كرّر نفس الخطوة مع **SHA256** بزرار SHA256.

4. نزّل google-services.json من جديد

- من نفس الصفحة في إعدادات Firebase، نزّل ملف `google-services.json` الجديد وحطه في المسار `android/app/` مكان الملف القديم.
- ⚠️ **تنبيه هام للفريق**: الـ SHA بتاع الـ debug بيختلف من جهاز لجهاز. كل مطوّر في الفريق لازم يطلع الـ SHA بتاعه ويضيفه في Firebase (المشروع بيقبل أكتر من واحد). ولما تطلّع نسخة release ضيف الـ SHA بتاعها كمان.

الجزء 5: كود Flutter (ملف social_auth.dart)

اعمل ملف `social_auth.dart` وحط فيه الكود التالي:

lib/social_auth.dart (Dart) Copy

```
import 'package:cloud_firestore/cloud_firestore.dart';
import 'package:dio/dio.dart';
import 'package:firebase_auth/firebase_auth.dart';
import 'package:google_sign_in/google_sign_in.dart';

const webClientId = 'حط-الـ-Web-client-ID-هنا';

final socialDio = Dio(BaseOptions(
  baseUrl: 'https://social-gateway.<subdomain>.workers.dev',
  validateStatus: (s) => s != null && s < 500,
));

bool _initialized = false;

/// بيعمل Google sign-in ويرجّع Firebase ID token
Future<String> signInWithGoogle() async {
  if (!_initialized) {
    await GoogleSignIn.instance.initialize(serverClientId: webClientId);
    _initialized = true;
  }
  final account = await GoogleSignIn.instance.authenticate();
  final credential = GoogleAuthProvider.credential(
    idToken: account.authentication.idToken,
  );
  final userCred = await FirebaseAuth.instance.signInWithCredential(credential);
  return (await userCred.user!.getIdToken())!;
}

/// لو لازم يعمل register بيرجّع بيانات الحساب لو موجود، أو null
Future<Map<String, dynamic>?> socialLogin(String idToken) async {
  final res = await socialDio.post('/social/login', data: {'token': idToken});
  if (res.statusCode == 200) return res.data;
  if (res.statusCode == 404) return null;
```

```
  throw Exception('Login failed: ${res.data}');
}

/// بعد ما تجمع كل البيانات الـ flow الـ register ابعته في آخر
Future<Map<String, dynamic>> socialRegister(
    String idToken, Map<String, dynamic> profile) async {
  final res = await socialDio
      .post('/social/register', data: {'token': idToken, 'profile': profile});
  if (res.statusCode != 200 && res.statusCode != 201) {
    throw Exception('Register failed: ${res.data}');
  }
  final email = FirebaseAuth.instance.currentUser!.email!.toLowerCase();
  await FirebaseFirestore.instance
      .collection('social_users')
      .doc(email)
      .set({'provider': 'google'});
  return res.data;
}
```

- `webClientId` : الـ Web client ID اللي نسخته من Firebase في الجزء الأول.
- `baseUrl` : لينك الـ Worker اللي طلعلك بعد الـ deploy.
- الكود ده مكتوب لحزمة `google_sign_in` نسخة 7 أو أحدث. لو نسختك أقدم الكود بيختلف.
- عميل Dio الأساسي بتاعك (اللي بيكلم الـ backend القديم) يفضل بـ Dio تاني منفصل بـ `https://fitness.elevateegy.com/api/v1`. أما `socialDio` فهو مخصص للـ login والـ register بالـ social بس.

للـ **release build**: اتأكد إن ملف `android/app/src/main/AndroidManifest.xml` فيه صلاحية الإنترنت:

AndroidManifest.xml (XML) Copy

```
<uses-permission android:name="android.permission.INTERNET"/>
```

الجزء 6: الـ Flow الكامل داخل التطبيق

1. Login بالـ Google

Dart (Flutter Flow) Copy

```
final token = await signInWithGoogle();
final result = await socialLogin(token);
if (result != null) {
  // وكمّل للتطبيق result['token'] الحساب موجود: خزّن
} else {
  // حساب جديد: ابدأ شاشات الـ register
}
```

2. Register (استكمال بيانات الحساب الجديد)

- خد الاسم والإيميل من Firebase مباشرة عبر: `FirebaseAuth.instance.currentUser` (الحقلين `displayName` و `email`).

- اجمع باقي البيانات المطلوبة (النوع، الطول، الوزن، العمر، الهدف، مستوى النشاط) في شاشات الـ flow.
- في آخر خطوة بعد جمع البيانات استدعِ `socialRegister` :

Dart (Flutter Flow) Copy

```
final data = await socialRegister(token, {
  'firstName': '...',
  'lastName': '...',
  'gender': 'male',
  'height': 170,
  'weight': 70,
  'age': 25,
  'goal': 'Gain weight',
  'activityLevel': 'level1',
});
// data['token'] هو token الـ backend
```

ملاحظة هامة: قيم `goal` و`activityLevel` لازم تطابق القيم النصية اللي الـ backend القديم بيقبلها بالظبط.

3. Forgot Password (على الـ backend القديم، من غير الـ Worker)

قبل استدعاء أول endpoint (اللي بيبعت كود الـ OTP للإيميل)، افحص الإيميل في Firestore الأول:

Dart (Flutter Flow) Copy

```
final doc = await FirebaseFirestore.instance
    .collection('social_users').doc(email.toLowerCase()).get();
if (doc.exists) {
  // اعرض: "الحساب ده بيدخل بـ Google، استخدم الزرار"
  // وامنع الطلب
  return;
}
// لو مش موجود: كمّل إرسال OTP ← التحقق ← تغيير الباسورد
```